Verifying an Election Majority over Equality-Only Ballots: A Divide-and-Conquer Approach

Sridhar Kumar
Department of Computer Science
University of Florida
Gainesville, FL, USA
f.sridharkumar@ufl.edu

Daen P Khan
Department of Computer Science
University of Florida
Gainesville, FL, USA
dpatan@ufl.edu

***Abstract*—Certifying an election requires deciding whether any candidate received more than half of the ballots. When ballots are encrypted, the tallying authority may only be able to test whether two ballots are for the same candidate, which rules out sorting or keeping a tally. We model this task as finding the majority element of a sequence under an equality oracle and give a divide-and-conquer algorithm that finds the majority of each half recursively and then recounts at most two finalists. We prove the algorithm correct and show that it runs in $O(n \log n)$ time. Experiments on synthetic elections of up to ten million ballots confirm the predicted growth.**

***Index Terms*—divide and conquer, majority element, election tallying, encrypted ballots, equality oracle, recurrence analysis**

I. Introduction

Many election rules require a candidate to win an *absolute majority*, more than half of all ballots cast, to be declared the winner outright; otherwise a runoff is held. Deciding whether such a majority exists, and for whom, is therefore a basic step in certifying a result.

Counting paper ballots is simple. Verifiable voting systems, however, often keep ballots encrypted during tallying so that no official learns how anyone voted [1]. Because the encryption is probabilistic, two ballots for the same candidate look different, so encrypted ballots can be neither tallied nor sorted. What remains is a *plaintext equivalence test* [2], which reveals only whether two ballots are for the same candidate.

This paper studies how to decide whether an absolute majority exists using only such equality tests. Our contributions are:

- An abstraction of the task as majority finding under an equality oracle (Section III).
- A divide-and-conquer algorithm, described formally and as a counting procedure (Sections IV and VII).
- A proof of correctness and an $O(n \log n)$ running-time analysis (Sections V and VI).
- An experimental validation on synthetic elections (Section VIII).

II. Related Work

The best-known algorithm for the majority problem is the Boyer–Moore voting algorithm (MJRTY) [3]. It cancels out pairs of ballots for different candidates in one pass and verifies the surviving candidate in a second, using only equality tests and $O(n)$ time. Misra and Gries [4] generalized this idea to find every value occurring more than n/k times, which underlies modern streaming "heavy hitter" summaries. We use Boyer–Moore as a baseline; the divide-and-conquer approach is asymptotically slower but splits naturally into independent sub-tallies, matching how ballots are divided among counting teams.

On the voting side, Helios [1] is a deployed web-based voting system that keeps ballots encrypted and lets anyone audit the tally. The plaintext equivalence test of Jakobsson and Juels [2] lets parties holding a decryption key jointly decide whether two ciphertexts encrypt the same message without revealing the message, which is exactly the equality oracle in our model.

Our analysis uses standard tools for divide-and-conquer algorithms [5], in particular the Master Theorem for recurrences of the form $T(n) = aT(n/b) + f(n)$, introduced in general form by Bentley, Haken, and Saxe [6].

III. Problem Formulation

A. Real-World Context

An election authority holds n encrypted ballots, each cast for one of an unknown number of candidates. The authority must certify either that some candidate received more than $n/2$ ballots, and which one, or that no candidate did and a runoff is needed. Each comparison of two ballots is a cryptographic protocol run jointly by the trustees, so it is the expensive operation; the authority wants to use as few comparisons as possible and to split the work among several counting teams.

B. Mathematical Abstraction

Definition 1 (Ballot sequence): Let U be a set of possible ballot values (candidates), whose size need not be known. A ballot sequence is $B = (b_0, b_1, \ldots, b_{n-1})$ with each $b_i \in U$. For indices $0 \le lo < hi \le n$ we write $B[lo..hi)$ for the contiguous subsequence $(b_{lo}, \ldots, b_{hi-1})$, of length $hi - lo$.

Definition 2 (Equality oracle): The only operation on ballot values is $\textsc{Eq} : U \times U \to \{\text{true}, \text{false}\}$, with $\textsc{Eq}(x, y) = \text{true}$ if and only if $x = y$. There is no order on U and no way to map a value to an index or hash, so sorting and tally tables are unavailable.

Definition 3 (Count and majority): For a sequence S and a value x, let $\mathrm{cnt}_S(x)$ be the number of positions of S holding x. A value x is the *majority* of S if $\mathrm{cnt}_S(x) > |S|/2$.

A sequence has at most one majority: two distinct majorities would together occupy more than $|S|$ positions. Since $\mathrm{cnt}_S(x)$ is an integer, $\mathrm{cnt}_S(x) > |S|/2$ holds exactly when $\mathrm{cnt}_S(x) > \lfloor |S|/2 \rfloor$.

Problem (Majority Verification). *Input:* a ballot sequence B of length $n \geq 1$, accessible only through EQ. *Output:* the majority of B if one exists, and the symbol $\perp$ ("no majority") otherwise. *Cost:* the number of calls to EQ, together with the running time.

Tree view. Recursively halving B produces a full binary tree whose root is B, whose internal nodes are contiguous segments $B[lo..hi)$ of length at least 2, and whose n leaves are the single ballots. Each internal node has two children, the left and right halves of its segment, so the tree has exactly $n-1$ internal nodes. The algorithm below computes, bottom-up, the majority (or $\perp$) of every node of this tree.

IV. Algorithm Design

A. Divide-and-Conquer Strategy

The algorithm rests on one observation, proved as Lemma 1: if a candidate is the majority of a sequence, it must be the majority of at least one of the two halves. Therefore, once the majority of each half is known, the only possible majority of the whole is one of those two values, and checking each of them takes a single pass. Algorithm 1 implements the three steps:

- **Divide:** split $B[lo..hi)$ at $mid = \lfloor (lo + hi)/2 \rfloor$.
- **Conquer:** recursively compute the majority ℓ of the left half and r of the right half (each possibly $\perp$).
- **Combine:** if both are $\perp$, return $\perp$; if both exist and are equal, return that value; otherwise count each existing finalist over the whole segment and return the one, if any, that exceeds half.

A finalist is carried as a representative ballot from its half, so comparing two finalists, or a finalist against a ballot, is itself one call to EQ.

The full problem is solved by the call MAJORITYDC$(B, 0, n)$.

B. Worked Example

Consider eight ballots $B = (A, A, B, A \mid B, A, C, A)$.

- Left half (A, A, B, A): its halves (A, A) and (B, A) return A and $\perp$, and recounting A gives $3 > 2$, so the left half returns A.
- Right half (B, A, C, A): its halves (B, A) and (C, A) both return $\perp$, so the right half returns $\perp$ without any counting.
- Root: the finalists are A and $\perp$. Recounting A over all eight ballots gives $5 > 4$, so the algorithm returns A.

Algorithm 1 Divide-and-conquer majority verification

Require: ballot sequence B, indices $0 \leq lo < hi \leq |B|$
Ensure: majority of $B[lo..hi)$, or $\perp$ if none exists

```
1:  function MajorityDC(B, lo, hi)
2:      if hi − lo = 1 then
3:          return b_lo                                  ▷ base case
4:      end if
5:      mid ← ⌊(lo + hi)/2⌋                              ▷ divide
6:      ℓ ← MajorityDC(B, lo, mid)                       ▷ conquer
7:      r ← MajorityDC(B, mid, hi)
8:      if ℓ = ⊥ and r = ⊥ then                          ▷ combine
9:          return ⊥
10:     end if
11:     if ℓ ≠ ⊥ and r ≠ ⊥ and Eq(ℓ, r) then
12:         return ℓ
13:     end if
14:     for each x ∈ (ℓ, r) with x ≠ ⊥ do
15:         if Count(B, lo, hi, x) > ⌊(hi − lo)/2⌋ then
16:             return x
17:         end if
18:     end for
19:     return ⊥
20: end function

21: function Count(B, lo, hi, x)
22:     c ← 0
23:     for i ← lo to hi − 1 do
24:         if Eq(b_i, x) then
25:             c ← c + 1
26:         end if
27:     end for
28:     return c
29: end function
```

C. Implementation Notes

The recursion works on index ranges of the original array, so no ballots are copied. In our Java implementation (see Code Availability), each ballot is stored as an integer, but every inspection of ballot values goes through a single method `same(a, b)` that performs one equality test and increments a counter. This keeps the implementation faithful to the equality-only model and lets the experiments report the exact number of oracle calls.

V. Complexity Analysis

A. Number of Equality Tests

Theorem 1: On $n \geq 2$ ballots, MAJORITYDC makes at most $2n\lceil \log_2 n \rceil + n - 1$ calls to EQ.

Proof: Leaves (single ballots) make no calls. Consider an internal node of the recursion tree whose segment has length $s \geq 2$. Its combine step makes at most one call comparing ℓ with r, and at most two calls to COUNT, each making exactly s calls. So the node makes at most $2s + 1$ calls. The tree has $n - 1$ internal nodes, contributing at most $n - 1$ in total for the "$+1$" terms.

For the $2s$ terms, group internal nodes by depth. The segments at any one depth are disjoint pieces of B, so their lengths sum to at most n. A node at depth d has length at most $\lceil n/2^d \rceil$, and an internal node has length at least 2; since $\lceil n/2^d \rceil \geq 2$ requires $2^d < n$, internal nodes occur only at depths $0, 1, \ldots, \lceil \log_2 n \rceil - 1$, which is $\lceil \log_2 n \rceil$ depths. Summing $2s$ over these depths gives at most $2n\lceil \log_2 n \rceil$. Adding the two parts proves the bound. ∎

B. Running Time

Let $T(n)$ be the worst-case running time on n ballots. The divide step takes constant time, the two recursive calls are on $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$ ballots, and the combine step takes at most two linear scans. Hence, for a constant $c > 0$,

$$T(n) \leq T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + c\,n, \qquad T(1) = O(1). \quad (1)$$

Ignoring floors and ceilings, which does not affect the asymptotic solution [5], this is $T(n) = aT(n/b) + f(n)$ with $a = 2$, $b = 2$, and $f(n) = \Theta(n)$. Since $n^{\log_b a} = n^{\log_2 2} = n$ and $f(n) = \Theta(n^{\log_b a})$, Case 2 of the Master Theorem [5], [6] gives

$$T(n) = O(n \log n). \quad (2)$$

This agrees with Theorem 1, since the work per call is proportional to the number of equality tests plus a constant.

The bound is an upper bound on the worst case; some inputs are cheaper. If all ballots are identical, every combine step makes a single agreement test, so the total is $n - 1$ tests and $\Theta(n)$ time. Likewise, when a candidate holds a large share, halves usually agree and recounts become rare, and when no candidate is close to half, halves usually both return $\perp$ and stop immediately. Section VIII observes both effects.

C. Space

The algorithm needs no auxiliary arrays. The recursion depth is $\lceil \log_2 n \rceil$, and each stack frame stores a constant number of indices and finalists, so the extra space is $O(\log n)$ beyond the $O(n)$ input.

VI. Proof of Correctness

Throughout, let $S = L \circ R$ denote a sequence S split into a left part L and a right part R, so that $|S| = |L| + |R|$ and $\mathrm{cnt}_S(x) = \mathrm{cnt}_L(x) + \mathrm{cnt}_R(x)$ for every value x.

Lemma 1 (Majority split): If x is the majority of $S = L \circ R$, then x is the majority of L or the majority of R (or both).

Proof: Suppose, for contradiction, that x is the majority of neither part. Then $\mathrm{cnt}_L(x) \leq |L|/2$ and $\mathrm{cnt}_R(x) \leq |R|/2$, so $\mathrm{cnt}_S(x) = \mathrm{cnt}_L(x) + \mathrm{cnt}_R(x) \leq (|L| + |R|)/2 = |S|/2$, contradicting that x is the majority of S. ∎

Lemma 2 (Agreement): If x is the majority of both L and R, then x is the majority of $S = L \circ R$.

Proof: $\mathrm{cnt}_S(x) = \mathrm{cnt}_L(x) + \mathrm{cnt}_R(x) > |L|/2 + |R|/2 = |S|/2$. ∎

Theorem 2: For every ballot sequence B and indices $0 \leq lo < hi \leq |B|$, MajorityDC(B, lo, hi) terminates and returns the majority of $B[lo..hi)$ if one exists, and $\perp$ otherwise.

Proof: We use strong induction on the segment length $m = hi - lo \geq 1$. Write $S = B[lo..hi)$, $L = B[lo..mid)$, and $R = B[mid..hi)$.

Base case ($m = 1$). The single ballot b_{lo} appears $1 > 1/2$ times, so it is the majority of S, and the algorithm returns it.

Inductive step ($m \geq 2$). Assume the claim holds for all lengths smaller than m. Since $m \geq 2$, both $|L| = \lfloor m/2 \rfloor$ and $|R| = \lceil m/2 \rceil$ are at least 1 and at most $m - 1$. By the induction hypothesis, both recursive calls terminate, ℓ is the majority of L or $\perp$ if L has none, and r is the majority of R or $\perp$ if R has none. We consider the three ways the combine step can end.

Case 1: $\ell = r = \perp$. Neither half has a majority, so by Lemma 1 no value is the majority of S. Returning $\perp$ is correct.

Case 2: $\ell \neq \perp$, $r \neq \perp$, and $\ell = r$. The value ℓ is the majority of both halves, so by Lemma 2 it is the majority of S. Returning ℓ is correct.

Case 3: otherwise. Let K be the set of non-$\perp$ values among ℓ and r; it is nonempty and has at most two elements. By Lemma 1, every majority of S is the majority of L or of R, so it lies in K. For each $x \in K$, Count examines every position of S once and returns exactly $\mathrm{cnt}_S(x)$, and the test $\mathrm{cnt}_S(x) > \lfloor m/2 \rfloor$ holds exactly when x is the majority of S. If S has a majority, it is in K and passes the test, and since the majority is unique, no other value in K can pass, so the algorithm returns it. If S has no majority, no value in K passes, and the algorithm returns $\perp$. In both situations the result is correct.

The combine step performs at most two loops of m iterations, so the call terminates. This completes the induction; taking $lo = 0$ and $hi = n$ shows that MajorityDC$(B, 0, n)$ solves Majority Verification. ∎

VII. The Algorithm as an Election Procedure

In the language of the election, the algorithm is a hierarchical counting procedure that never reveals more than "same candidate or not" about any pair of ballots.

The chief election official splits the stack of sealed ballots into two roughly equal piles and hands each pile to a counting team. Each team splits its pile again and delegates, until every pile holds a single ballot, whose "leader" is simply the candidate on it. Teams then report upward. Each team receives two reports, one from each sub-team, stating either a provisional leader of that sub-pile (represented by one of its ballots) or "no leader."

- If **both sub-teams report no leader**, the team reports no leader without examining anything: a candidate who failed to reach half of both sub-piles cannot reach half of the combined pile.
- If **both report the same leader**, confirmed by one equivalence test between their representative ballots, that candidate leads the combined pile: more than half of each sub-pile adds up to more than half of the whole.
- Otherwise, at most two **finalists** remain. The team runs a recount, comparing each finalist's representative ballot

TABLE I
SYNTHETIC ELECTION SCENARIOS (FIVE CANDIDATES)

Scenario	Candidate 0 receives	Outcome
LANDSLIDE	$\lfloor 0.6n \rfloor$ ballots	0 wins
NARROW	$\lfloor n/2 \rfloor + 1$ ballots	0 wins by one
TIE	$\lfloor n/2 \rfloor$ ballots	no majority
FRAGMENTED	about $n/5$ (uniform)	no majority

TABLE II
MEDIAN RUNNING TIME OF MAJORITYDC (MS)

n	LANDSLIDE	NARROW	TIE	FRAGM.
10^3	0.010	0.012	0.012	0.009
10^4	0.103	0.142	0.139	0.099
10^5	1.322	1.760	1.873	1.092
10^6	13.745	20.057	20.733	11.602
10^7	132.161	215.525	218.795	111.619

against every ballot in its pile, and reports whichever finalist exceeds half, or no leader.

The chief official's final report is the certified outcome: either the candidate holding an absolute majority, or the conclusion that a runoff is required. No candidate can be overlooked, because a true majority winner always survives as a finalist in at least one sub-pile at every level. Teams at the same level work on disjoint piles and never need to coordinate, so the procedure parallelizes naturally across counting rooms or precincts.

VIII. EXPERIMENTAL VALIDATION

A. Implementation and Correctness Testing

We implemented Algorithm 1 in Java, together with two baselines used for testing and comparison: a brute-force method that recounts every ballot's candidate ($O(n^2)$), and the Boyer–Moore voting algorithm [3] ($O(n)$). Before timing, we checked that the three methods agree on 210,160 inputs, including every sequence of length up to 8 over three candidates. All tests passed, and no run exceeded the bound of Theorem 1.

B. Synthetic Elections and Setup

Each synthetic election has five candidates and randomly shuffled ballots (Table I). TIE and NARROW are the hardest cases, since the leader holds about half of every large sub-pile.

We tested thirteen sizes from $n = 10^3$ to 10^7, with five random trials per size and scenario (brute force only up to $5 \cdot 10^4$). After a JVM warm-up, each trial was repeated until at least 20 ms had elapsed, and we report the median of the five trials. Experiments ran on a two-vCPU Linux virtual machine with OpenJDK 21.

C. Results

Running time matches $n \log n$. On the worst-case TIE elections, a fitted $c\, n \log_2 n$ curve explains the measured times with $R^2 = 0.9998$ (Fig. 1a); ten million ballots take about 219 ms (Table II).

Normalized time is nearly constant. Divided by $n \log_2 n$, the running time for NARROW and TIE stays between about 0.9 and 1.2 ns across four orders of magnitude (Fig. 1b). LANDSLIDE and FRAGMENTED elections fall below this level, as predicted in Section V, because their halves usually agree or both report no leader, so fewer recounts are needed.

Equality tests stay within the bound. For TIE elections the algorithm uses about 0.6–$0.7\, n \log_2 n$ equality tests, roughly 30% of the proven bound, and no trial exceeded it (Fig. 1c). Unlike running time, this count does not depend on the machine.

Comparison with baselines. Brute force grows quadratically and takes about 1.8 s at $n = 5 \cdot 10^4$, versus 0.89 ms for divide and conquer; Boyer–Moore is about $3.8\times$ faster at $n = 10^7$ (Fig. 1d).

IX. DISCUSSION

A. When Divide and Conquer Fits

For a purely sequential tally, Boyer–Moore is faster. The divide-and-conquer algorithm's strengths are structural: its recursive calls are independent, so sub-piles can be verified by separate teams or processors, and every intermediate report states the majority of a specific pile, which supports auditing at the level of individual counting rooms.

B. Limitations

The experiments use synthetic, uniformly shuffled ballots; real ballot orders may be correlated with precinct, which can change how often recounts occur but not the worst-case bound. Running times are machine-dependent, and we model the equality test as a unit-cost operation rather than implementing the cryptographic protocol.

X. CONCLUSION

We gave a divide-and-conquer algorithm for verifying an election majority when ballots can only be compared for equality, based on the fact that a majority of the whole must be a majority of one of its halves. We proved it correct, showed that it runs in $O(n \log n)$ time, and confirmed the analysis experimentally. Future work includes a parallel implementation and an evaluation with a real plaintext equivalence test.

REFERENCES

[1] B. Adida, "Helios: Web-based open-audit voting," in *Proc. 17th USENIX Security Symposium*, San Jose, CA, USA, 2008.

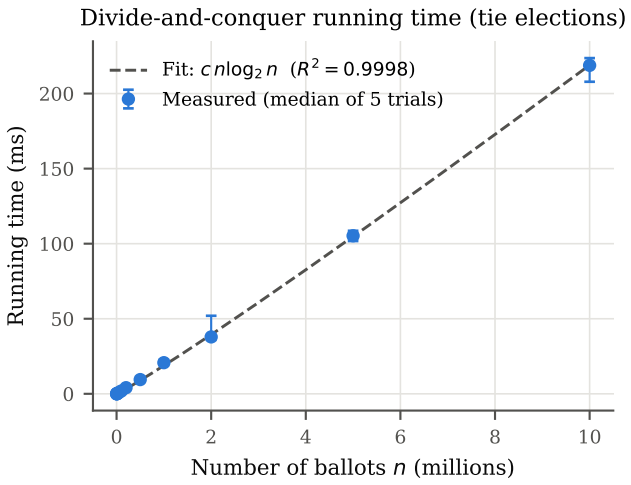
[2] M. Jakobsson and A. Juels, "Mix and match: Secure function evaluation via ciphertexts," in *Advances in Cryptology – ASIACRYPT 2000*, ser. Lecture Notes in Computer Science, vol. 1976. Berlin, Germany: Springer, 2000, pp. 162–177.

[3] R. S. Boyer and J. S. Moore, "MJRTY—A fast majority vote algorithm," in *Automated Reasoning: Essays in Honor of Woody Bledsoe*, R. S. Boyer, Ed. Dordrecht, The Netherlands: Kluwer Academic Publishers, 1991, pp. 105–117.

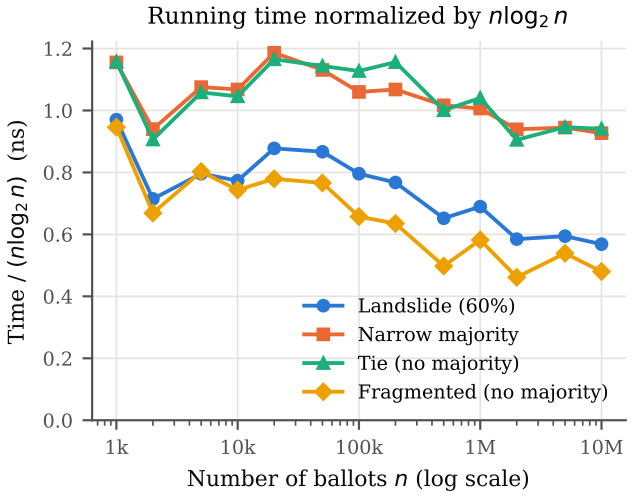
[4] J. Misra and D. Gries, "Finding repeated elements," *Science of Computer Programming*, vol. 2, no. 2, pp. 143–152, 1982.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

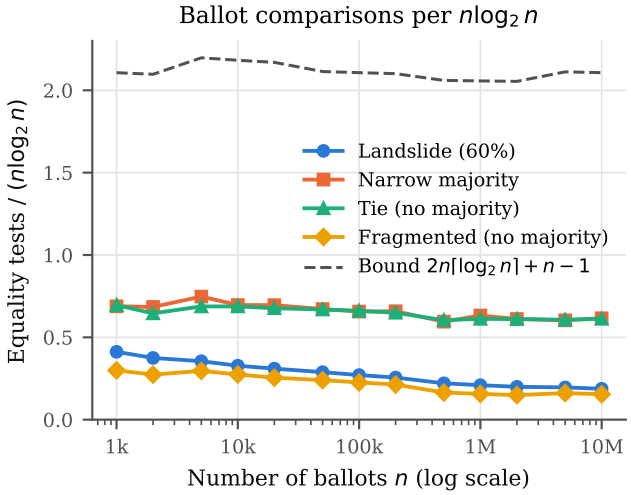
[6] J. L. Bentley, D. Haken, and J. B. Saxe, "A general method for solving divide-and-conquer recurrences," *ACM SIGACT News*, vol. 12, no. 3, pp. 36–44, 1980.



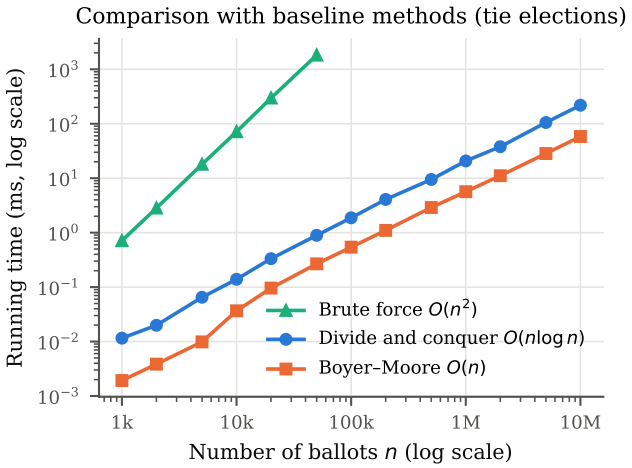
(a) Running time on TIE elections (median of five trials; bars show minimum and maximum) with the least-squares fit $c\,n\log_2 n$, $c \approx 0.94$ ns.



(b) Running time divided by $n \log_2 n$. A flat curve indicates $\Theta(n \log n)$ growth.



(c) Equality tests divided by $n \log_2 n$, with the bound of Theorem 1 (dashed).



(d) Running time of the three methods on TIE elections (log–log scale).

Fig. 1. Experimental results for MAJORITYDC on synthetic elections of 10^3 to 10^7 ballots.

APPENDIX A
RUNTIME VALIDATION CODE

The following Java source files perform correctness checks, generate the synthetic elections, measure repeated timing trials, and write the CSV data used for the experimental figures. The majority algorithm is included because it is the implementation called by the benchmark.

Listing 1. Divide-and-conquer benchmark and validation harness

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Locale;
import java.util.SplittableRandom;

/**
 * Benchmark.java
 *
 * Synthetic election generator, correctness tests and running-time
 * experiments for the divide-and-conquer majority algorithm.
 *
 * Usage:
 *   javac MajorityVote.java Benchmark.java
 *   java Benchmark verify               # correctness tests
 *   java Benchmark bench results.csv    # timing experiments
 */
public final class Benchmark {

    /** Number of candidates on the synthetic ballot. */
    static final int CANDIDATES = 5;

    /** Ballot counts used in the timing experiments. */
    static final int[] SIZES = {
        1_000, 2_000, 5_000, 10_000, 20_000, 50_000, 100_000, 200_000,
        500_000, 1_000_000, 2_000_000, 5_000_000, 10_000_000
    };

    /** Brute force is O(n^2), so it is only run on the smaller elections. */
    static final int BRUTE_FORCE_MAX_N = 50_000;

    static final int TRIALS = 5;

    /** Repeat a measurement until at least this much time has elapsed. */
    static final long MIN_MEASURE_NANOS = 20_000_000L;  // 20 ms

    /** Kinds of synthetic elections. */
    enum Scenario {
        LANDSLIDE,    // candidate 0 receives 60% of the ballots
        NARROW,       // candidate 0 receives floor(n/2) + 1 ballots (smallest
       majority)
        TIE,          // candidate 0 receives exactly floor(n/2) ballots: no majority
        FRAGMENTED    // every ballot is uniform over the 5 candidates: no majority
```

```
    }

    // ------------------------------------------------------------------
    // Synthetic data
    // ------------------------------------------------------------------

    /** Generates n shuffled ballots for the given scenario. */
    static int[] generateBallots(int n, Scenario scenario, long seed) {
        SplittableRandom rng = new SplittableRandom(seed);
        int[] ballots = new int[n];
        int leaderVotes;
        switch (scenario) {
            case LANDSLIDE:  leaderVotes = (int) (0.6 * n); break;
            case NARROW:     leaderVotes = n / 2 + 1;       break;
            case TIE:        leaderVotes = n / 2;           break;
            default:         leaderVotes = -1;              break;
        }
        if (leaderVotes < 0) {
            for (int i = 0; i < n; i++) ballots[i] = rng.nextInt(CANDIDATES);
        } else {
            leaderVotes = Math.min(leaderVotes, n);
            for (int i = 0; i < leaderVotes; i++) ballots[i] = 0;
            // Remaining ballots go to the other candidates (ids 1..4).
            for (int i = leaderVotes; i < n; i++) ballots[i] = 1 + rng.nextInt(
       CANDIDATES - 1);
        }
        // Fisher-Yates shuffle so the leader's ballots are spread throughout.
        for (int i = n - 1; i > 0; i--) {
            int j = rng.nextInt(i + 1);
            int t = ballots[i]; ballots[i] = ballots[j]; ballots[j] = t;
        }
        return ballots;
    }

    /** Upper bound on equality tests: 2 n ceil(log2 n) + (n - 1). */
    static long comparisonBound(int n) {
        if (n <= 1) return 0;
        int ceilLog = 32 - Integer.numberOfLeadingZeros(n - 1);
        return 2L * n * ceilLog + (n - 1);
    }

    // ------------------------------------------------------------------
    // Correctness tests
    // ------------------------------------------------------------------

    static void verify() {
        MajorityVote mv = new MajorityVote();
        long tests = 0, withMajority = 0;

        // 1. Exhaustive: every ballot sequence of length 1..8 over 3 candidates.
        for (int n = 1; n <= 8; n++) {
            int total = (int) Math.pow(3, n);
            for (int code = 0; code < total; code++) {
                int[] b = new int[n];
                int c = code;
                for (int i = 0; i < n; i++) { b[i] = c % 3; c /= 3; }
                if (check(mv, b)) withMajority++;
                tests++;
            }
        }
        System.out.printf("Exhaustive tests (n <= 8, 3 candidates): %d passed%n",
       tests);

        // 2. Random small elections with 1..4 candidates.
        SplittableRandom rng = new SplittableRandom(2026);
        long randomTests = 200_000;
        for (long t = 0; t < randomTests; t++) {
            int n = 1 + rng.nextInt(64);
            int k = 1 + rng.nextInt(4);
            int[] b = new int[n];
            for (int i = 0; i < n; i++) b[i] = rng.nextInt(k);
            if (check(mv, b)) withMajority++;
        }
        tests += randomTests;
        System.out.printf("Random tests (n <= 64): %d passed%n", randomTests);

        // 3. Every scenario at several sizes, including odd sizes.
        int scenarioTests = 0;
        for (Scenario s : Scenario.values()) {
            for (int n : new int[] {1, 2, 3, 7, 100, 1001, 4096, 10_007}) {
                for (long seed = 0; seed < 10; seed++) {
                    int[] b = generateBallots(n, s, seed);
                    int winner = checkWinner(mv, b);
                    if (winner != MajorityVote.NO_MAJORITY) withMajority++;
                    // The generator must produce the outcome each scenario promises
       .
                    boolean leaderMustWin = s == Scenario.NARROW
                            || (s == Scenario.LANDSLIDE && (int) (0.6 * n) > n / 2);
                    if (leaderMustWin && winner != 0) {
                        throw new AssertionError(s + ": candidate 0 should win, n="
       + n);
                    }
                    // With n even, a tie leaves n/2 ballots for everyone else,
                    // so nobody can exceed n/2.
                    if (s == Scenario.TIE && n % 2 == 0 && winner != MajorityVote.
       NO_MAJORITY) {
                        throw new AssertionError("TIE produced a majority, n=" + n);
                    }
                    scenarioTests++;
                }
            }
        }
        tests += scenarioTests;
        System.out.printf("Scenario tests: %d passed%n", scenarioTests);
        System.out.printf("ALL %d TESTS PASSED (%d inputs had a majority winner)%n",
                tests, withMajority);
    }

    /**
     * Checks D&C against brute force and Boyer-Moore, and checks the
     * equality-test bound. Returns true if the input has a majority.
     */
    static boolean check(MajorityVote mv, int[] b) {
        return checkWinner(mv, b) != MajorityVote.NO_MAJORITY;
    }

    /** Same checks as check(), returning the winner (or NO_MAJORITY). */
    static int checkWinner(MajorityVote mv, int[] b) {
        mv.resetComparisons();
        int dc = mv.majorityDC(b);
        long comps = mv.getComparisons();
        int bf = mv.majorityBruteForce(b);
        int bm = mv.majorityBoyerMoore(b);
        if (dc != bf || dc != bm) {
            throw new AssertionError("Mismatch on " + java.util.Arrays.toString(b)
                    + ": dc=" + dc + " brute=" + bf + " boyerMoore=" + bm);
        }
        if (comps > comparisonBound(b.length)) {
            throw new AssertionError("Comparison bound exceeded for n=" + b.length
                    + ": " + comps + " > " + comparisonBound(b.length));
        }
        return dc;
    }

    // ------------------------------------------------------------------
    // Timing experiments
    // ------------------------------------------------------------------

    interface Solver { int solve(MajorityVote mv, int[] b); }

    static volatile int sink;  // keeps the JIT from discarding results

    static void bench(String csvPath) throws IOException {
        MajorityVote mv = new MajorityVote();

        // Warm up the JIT before measuring anything.
        int[] warm = generateBallots(1_000_000, Scenario.NARROW, 99);
        for (int i = 0; i < 15; i++) {
            sink = mv.majorityDC(warm);
            sink = mv.majorityBoyerMoore(warm);
        }
        int[] warmSmall = generateBallots(5_000, Scenario.TIE, 99);
        for (int i = 0; i < 5; i++) sink = mv.majorityBruteForce(warmSmall);

        try (PrintWriter out = new PrintWriter(new FileWriter(csvPath))) {
            out.println("algorithm,scenario,n,trial,reps,time_ms,comparisons,winner"
       );
            for (Scenario s : Scenario.values()) {
                for (int n : SIZES) {
                    for (int trial = 0; trial < TRIALS; trial++) {
                        long seed = 1_000_003L * n + 31L * trial + s.ordinal();
                        int[] b = generateBallots(n, s, seed);

                        measure(out, mv, "DivideAndConquer", s, n, trial, b,
                                (m, x) -> m.majorityDC(x));
                        measure(out, mv, "BoyerMoore", s, n, trial, b,
                                (m, x) -> m.majorityBoyerMoore(x));
                        if (n <= BRUTE_FORCE_MAX_N) {
                            measure(out, mv, "BruteForce", s, n, trial, b,
                                    (m, x) -> m.majorityBruteForce(x));
                        }
                    }
                    System.out.printf("done %-10s n=%,d%n", s, n);
                    out.flush();
                }
            }
        }
    }

    static void measure(PrintWriter out, MajorityVote mv, String name, Scenario s,
                        int n, int trial, int[] b, Solver solver) {
        // One counted run for the comparison count and the winner.
        mv.resetComparisons();
        int winner = solver.solve(mv, b);
        long comps = mv.getComparisons();

        // Timed runs: repeat until at least MIN_MEASURE_NANOS have passed.
        int reps = 0;
        long start = System.nanoTime(), elapsed;
        do {
            sink = solver.solve(mv, b);
            reps++;
            elapsed = System.nanoTime() - start;
        } while (elapsed < MIN_MEASURE_NANOS);

        double ms = elapsed / 1e6 / reps;
        out.printf(Locale.US, "%s,%s,%d,%d,%d,%.6f,%d,%d%n",
                name, s, n, trial, reps, ms, comps, winner);
    }

    public static void main(String[] args) throws IOException {
        if (args.length >= 1 && args[0].equals("verify")) {
            verify();
        } else if (args.length >= 1 && args[0].equals("bench")) {
            bench(args.length >= 2 ? args[1] : "results.csv");
        } else {
            System.out.println("Usage: java Benchmark verify | bench [results.csv]")
       ;
        }
    }
}
```

Listing 2. Majority algorithm implementation

```
/**
 * MajorityVote.java
 *
 * Divide-and-conquer verification of an election majority.
 *
 * Model: ballots can only be tested for equality ("are these two ballots
```

```java
 * for the same candidate?"). No sorting and no tally table are used.
 * Each ballot is stored as a candidate id (int >= 0), but the algorithms
 * below only ever look at two ballots through same(), which counts every
 * equality test so the experiments can report it.
 *
 * A candidate x is the majority winner of n ballots if it appears on
 * more than n/2 of them. Methods return NO_MAJORITY (-1) when no such
 * candidate exists.
 */
public final class MajorityVote {

    /** Returned when no candidate holds more than half the ballots. */
    public static final int NO_MAJORITY = -1;

    /** Number of equality tests performed since the last reset. */
    private long comparisons = 0;

    public long getComparisons() { return comparisons; }
    public void resetComparisons() { comparisons = 0; }

    /** The only way the algorithms inspect ballots: one equality test. */
    private boolean same(int a, int b) {
        comparisons++;
        return a == b;
    }

    // ---------------------------------------------------------------------
    // Divide-and-conquer algorithm
    // ---------------------------------------------------------------------

    /** Majority winner of all ballots, or NO_MAJORITY. */
    public int majorityDC(int[] ballots) {
        if (ballots.length == 0) return NO_MAJORITY;
        return majorityDC(ballots, 0, ballots.length);
    }

    /**
     * Majority winner of ballots[lo..hi) (hi exclusive), or NO_MAJORITY.
     * Requires hi - lo >= 1.
     */
    private int majorityDC(int[] ballots, int lo, int hi) {
        // Base case: a single ballot is its own majority.
        if (hi - lo == 1) return ballots[lo];

        // Divide: split the ballots into two halves.
        int mid = (lo + hi) >>> 1;

        // Conquer: each half reports its own majority winner (or none).
        int left = majorityDC(ballots, lo, mid);
        int right = majorityDC(ballots, mid, hi);

        // Combine.
        // Neither half has a winner, so the whole cannot have one (Lemma 1).
        if (left == NO_MAJORITY && right == NO_MAJORITY) return NO_MAJORITY;

        // Both halves agree: that candidate holds more than half of each
        // half, hence more than half of the whole.
        if (left != NO_MAJORITY && right != NO_MAJORITY && same(left, right)) {
            return left;
        }

        // Otherwise recount the (at most two) finalists over the whole range.
        int half = (hi - lo) / 2;  // count > n/2  <=>  count > floor(n/2)
        if (left != NO_MAJORITY && count(ballots, lo, hi, left) > half) return left;
        if (right != NO_MAJORITY && count(ballots, lo, hi, right) > half) return
       right;
        return NO_MAJORITY;
    }

    /** Number of ballots in ballots[lo..hi) cast for candidate x. */
    private int count(int[] ballots, int lo, int hi, int x) {
        int c = 0;
        for (int i = lo; i < hi; i++) {
            if (same(ballots[i], x)) c++;
        }
        return c;
    }

    // ---------------------------------------------------------------------
    // Baselines (used for verification and comparison only)
    // ---------------------------------------------------------------------

    /** Brute force, O(n^2): recount every ballot's candidate. */
    public int majorityBruteForce(int[] ballots) {
        int n = ballots.length;
        for (int i = 0; i < n; i++) {
            if (count(ballots, 0, n, ballots[i]) > n / 2) return ballots[i];
        }
        return NO_MAJORITY;
    }

    /** Boyer-Moore voting algorithm, O(n): one pairing pass + one recount. */
    public int majorityBoyerMoore(int[] ballots) {
        int n = ballots.length;
        if (n == 0) return NO_MAJORITY;
        int candidate = ballots[0];
        int lead = 0;
        for (int i = 0; i < n; i++) {
            if (lead == 0) {
                candidate = ballots[i];
                lead = 1;
            } else if (same(ballots[i], candidate)) {
                lead++;
            } else {
                lead--;
            }
        }
        return count(ballots, 0, n, candidate) > n / 2 ? candidate : NO_MAJORITY;
    }
}
```

APPENDIX B
LLM USAGE

A. Tool and Scope

We used Claude (model Claude Opus 5.5, by Anthropic), accessed through the claude.ai web interface in October 2026. It was used to tackle the correctness of the problem, implement some of the Java code and plotting script, run the experiments, generate the figures, draft this report in LaTeX.

B. Prompts and Outputs

The LLM was asked in prompts to question the correctness of certain concepts. The prompts have been cleaned up but essentially they boil down to the following

1) **Question:** *Prove that if x is the majority of $L \circ R$, then x is the majority of L or of R. Make sure it holds when $|L| \neq |R|$.*
   Answer: Suppose x is the majority of neither part. Then $\mathrm{cnt}_L(x) \leq |L|/2$ and $\mathrm{cnt}_R(x) \leq |R|/2$, and adding gives $\mathrm{cnt}_S(x) \leq |S|/2$, a contradiction. The argument never uses $|L| = |R|$, so it holds for every split, including the $\lfloor m/2 \rfloor$ / $\lceil m/2 \rceil$ split used for odd m.
2) **Question:** *Show that a sequence has at most one majority, and that* $\mathrm{cnt} > m/2$ *is equivalent to* $\mathrm{cnt} > \lfloor m/2 \rfloor$ *for integer counts.*
   Answer: Two distinct values occupy disjoint positions, so $\mathrm{cnt}_S(x) + \mathrm{cnt}_S(y) \leq |S|$; if both exceeded $|S|/2$ the sum would exceed $|S|$. For the threshold, the two conditions coincide when m is even. When m is odd, $m/2 = \lfloor m/2 \rfloor + \frac{1}{2}$, and an integer c satisfies $c > \lfloor m/2 \rfloor + \frac{1}{2}$ exactly when $c \geq \lfloor m/2 \rfloor + 1$, that is, when $c > \lfloor m/2 \rfloor$.
3) **Question:** *Is the case split on (ℓ, r) in Theorem 2 exhaustive?*
   Answer: Yes. Each of ℓ, r is either $\perp$ or a value, giving four combinations: both $\perp$ (Case 1); both values and equal (Case 2); both values and distinct (Case 3); exactly one $\perp$ (Case 3). Because EQ is an exact equality test, the algorithm's condition $\mathrm{EQ}(\ell, r)$ matches the proof's condition $\ell = r$.
4) **Question:** *In Case 3 with $\ell \neq r$ and both existing, can both finalists pass the count test?*
   Answer: No. Passing the test means $\mathrm{cnt}_S(x) > \lfloor m/2 \rfloor$, which by the previous answer means x is the majority of S, and a sequence has at most one majority. Hence the order in which the loop checks the finalists does not affect the result.
5) **Question:** *Is the case $\ell = \perp$, $r \neq \perp$ fully covered by the recount branch?*
   Answer: Yes. Here $K = \{r\}$. Since L has no majority, Lemma 1 implies that any majority of S must be the majority of R, which is r. The algorithm counts r over S and returns it exactly when it exceeds $\lfloor m/2 \rfloor$, and returns $\perp$ otherwise; both outcomes are correct. The loop

skips $\perp$, so no equality test is made on it. The case $\ell \neq \perp$, $r = \perp$ is symmetric.

6) **Question:** *Where exactly does the proof use $m \geq 2$?*
Answer: Since $mid = \lfloor (lo+hi)/2 \rfloor$ and lo is an integer, $|L| = mid - lo = \lfloor m/2 \rfloor$ and $|R| = \lceil m/2 \rceil$. For $m \geq 2$ we get $\lfloor m/2 \rfloor \geq 1$ and $\lceil m/2 \rceil \leq m - 1$, so both halves are non-empty (the recursive calls satisfy the precondition $lo < hi$) and strictly shorter than m (the induction hypothesis applies). For $m = 1$ one half would be empty, which is why $m = 1$ is the base case.
7) **Question:** *Prove an exact upper bound on the number of equality tests by counting the internal nodes of the recursion tree and bounding the work per level.*
Answer: An internal node with segment length s makes at most one agreement test and at most two counts of s tests each, so at most $2s + 1$ tests. The recursion tree is a full binary tree with n leaves, so it has $n - 1$ internal nodes, contributing at most $n - 1$ for the "$+1$" terms. Segments at the same depth are disjoint, so their lengths sum to at most n, and internal nodes occur at $\lceil \log_2 n \rceil$ depths (next answer). The total is at most $2n\lceil \log_2 n \rceil + n - 1$, as in Theorem 1.
8) **Question:** *Justify that internal nodes occur at only $\lceil \log_2 n \rceil$ depths when n is not a power of 2.*
Answer: By induction on depth, a node at depth d has length at most $\lceil n/2^d \rceil$, using $\lceil \lceil a \rceil / 2 \rceil = \lceil a/2 \rceil$. An internal node has length at least 2, and $\lceil n/2^d \rceil \geq 2$ holds exactly when $2^d < n$, that is, when $d \leq \lceil \log_2 n \rceil - 1$. For example, with $n = 5$ the internal nodes have lengths 5 (depth 0), 2 and 3 (depth 1), and 2 (depth 2), which is $\lceil \log_2 5 \rceil = 3$ depths.
9) **Question:** *Is applying the Master Theorem valid with floors and ceilings?*
Answer: Yes; Section 4.6.2 of [5] shows that the Master Theorem's conclusions still hold when the subproblem sizes are $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. A direct argument also avoids the issue: by Theorem 1 the algorithm makes at most $2n\lceil \log_2 n \rceil + n - 1$ equality tests, and each of its $2n - 1$ recursive calls does $O(1)$ work besides those tests, so $T(n) = O(n \log n)$.

C. Verification

The authors are responsible for the correctness of this report. The implementation was checked against two independent methods on 210,160 inputs, including every input of length up to 8 over three candidates, and the bound of Theorem 1 was checked on every test and benchmark run. Each bibliographic reference was checked against the publisher's or author's page.

Code Availability

The complete Java implementation (`MajorityVote.java` and `Benchmark.java`), the plotting script (`plot_results.py`), and the raw benchmark results (`results.csv`) are available in our repository. The complete code repository is available at github. To reproduce the results, run `javac MajorityVote.java Benchmark.java`, then `java Benchmark verify`, `java -Xmx3g Benchmark bench results.csv`, and `python3 plot_results.py results.csv figures`.